

Synthesizing Staged Updates between Verified Controllers

ANONYMOUS AUTHOR(S)

A system may have verified controllers for both its old and new versions while still lacking a safe way to move between them. Components may need separate replacement steps, and old and new requirements may need different periods of overlap. We present Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS): a local contract specifies component transfers, requirement lifetimes and initial obligations, and a compiler generates an update policy between the supplied controllers. The policy handles every reachable old entry and every permitted event outcome, safely completes the update, and then preserves the chosen new controller's behavior. We prove correspondence between local contracts and their generated games, and specialize established lazy reachability methods to return a policy with a progress certificate or a losing-region certificate. Complete examples characterize when merging update steps prevents completion and when it enables it. An MTSA implementation decides 26/27 adapted contracts within 1,200 s, compared with 17 for eager successors and 14 for full construction. Constructed families exhibit granularity effects; inherited inputs exhibit none. The result supports finite, quiescent handover under declared safety requirements; it does not guarantee continuous service or an elapsed-time bound.

CCS Concepts: • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: dynamic software updating, discrete-event controller synthesis, supervisory control, on-the-fly synthesis

ACM Reference Format:

Anonymous Author(s). 2027. Synthesizing Staged Updates between Verified Controllers. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

A verified controller governs one version. Updating requires reaching another while component states and active requirements change [3, 6, 14, 34]. The endpoints do not supply this intervening policy. We generate it while preserving the supplied new controller's behavior after handover.

Two choices make this more than a replacement order. Components may need separate replacement times: with one workpiece shared by two stations, an empty-station replacement guard prevents joint replacement, but moving the workpiece can enable separate replacements. Requirements also need separate lifetimes: continuous audit coverage needs new-start before old-stop, whereas mutually exclusive roles require the opposite order.

A fixed sequence cannot always handle different old states or multiple outcomes. The policy must respond to observations and preserve every environment outcome. Initialization must also retain obligations already due, such as inspection before departure. Safety alone permits an update to remain unfinished forever.

FG-DUCS compiles local choices into a policy for safe finite handover. The modeler supplies transfers, requirement lifetimes, monitor initializers and compatible endpoint states. The compiler generates mixed-state behavior and chooses a policy; neither a mixed-state graph nor a sequence is supplied. Modeling and transfer code remain the modeler's responsibility; effort reduction is unmeasured.

Existing DUCS generates update and subsequent control from global transfers and transition requirements [30, 31]; GR(1) updating generates a new controller and bridge [2]. Both express Cell with additional supplied structure (Section 3.4). FG-DUCS fixes the endpoints and generates the

intervening game from local transfers, independent boundaries and initializers. Its contribution is this construction and its guarantees, not greater general expressiveness or a new reachability algorithm.

Our contributions are:

C1: A local contract and its meaning. We define the generated game, prove local-history correspondence and selected-endpoint preservation, and give an invariant-initialization criterion (Sections 3 and 4.1).

C2: A policy with checkable evidence. Sound and complete lazy synthesis returns a completing policy and rank or a losing-region certificate (Section 5).

C3: Boundaries of update granularity. Examples and merging obstructions explain granularity-dependent feasibility, including a reverse example where finer updates lose (Sections 2 and 6.2).

C4: Implementation evidence. An MTSA implementation is checked against finite reference games and evaluated on 27 adapted contracts; constructed comparisons test the granularity mechanisms separately (Section 6). All unfavorable and unresolved outcomes are retained.

The target is a finite maintenance interval: controllable work may pause while pending environment events finish. Commands and handover require global uncontrollable quiescence. Continuous service and perpetual uncontrollable arrivals fall outside this contract; real-time bounds and execution conformance need additional assumptions.

Our public replication package [4], prepared for this paper, includes the cited Technical Appendix A–P and Supplement S1–S5 as Parts I and II of `supplementary_material.pdf`. Data Availability gives versioned links.

2 From a Complete Contract to a Completing Policy

Component granularity chooses what is replaced together; requirement granularity chooses which guarantees start or end together. Two contracts separate these choices.

2.1 Cell: Complete Local Input and Whole Policy

Consider a constructed two-station cell with exactly one workpiece [21, 31]. Both transfers require an empty station. A keeps its behavior; new B requires inspection before departure, while the old rule forbids inspection. Ordinary events move or inspect the workpiece. New-start before old-stop prevents an unprotected gap; a pending inspection waits for old-stop. Inspection can repeat safely forever.

Table 1 specifies every component, endpoint and monitor. The interval requires exactly one workpiece, which plant transitions preserve. All labels are controllable; new-start before old-stop is the only dependency. Transfers are $g_A = g_B = \{(e, e)\}$, where e means empty.

Endpoint tuples list A, B , controller and its requirement monitor. The old initial tuple is $x_A = (h, e, c_0, k)$ and its other reachable tuple is $x_B = (e, h, c_0, k)$; both are entries. The new initial tuple is $z_A = (h, e, c_0, c)$; its other reachable tuples are $y_B = (e, h, c_1, d)$ and $z_B = (e, h, c_0, c)$. Only $\{z_A, z_B\}$ are loadable targets: inspection must be clear at handover. The interval initializer records the holder in either all-old tuple. The new initializer admits all eight version-tagged one-workpiece tuples, installing d if B holds and c otherwise. These data specify the contract without an update sequence.

The generated policy. Figure 1’s 13 states and 11 edges cover both entries and targets. Each displayed event has one outcome; other controls are disabled. Both paths move the workpiece to enable empty-station replacements. The first also clears inspection debt before handover. Every

Table 1. Complete Cell components, controllers and monitors. o/n denote old/new versions; h/e mean holding/empty. Parentheses mark initial states. m_A, m_B move to A, B ; j inspects at B .

Object	States (initial)	Ordinary transitions
A^o, A^n	$h, e (h)$	$h \xrightarrow{m_B} e, e \xrightarrow{m_A} h$
B^o	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e$
B^n	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e, h \xrightarrow{j} h$
C^o	$c_0 (c_0)$	$c_0 \xrightarrow{m_A, m_B} c_0$
C^n	$c_0, c_1 (c_0)$	$c_0 \xrightarrow{m_A, j} c_0, c_0 \xrightarrow{m_B} c_1, c_1 \xrightarrow{j} c_0, c_1 \xrightarrow{m_B} c_1$
t_{one}	$A, B, \text{err} (A)$	$A \xrightarrow{m_B} B, B \xrightarrow{m_A} A, A \xrightarrow{m_A} \text{err}, B \xrightarrow{m_B} \text{err}$
t_{old}	$k, \text{err} (k)$	$k \xrightarrow{j} \text{err}$
t_{inspect}	$c, d, \text{err} (c)$	$c \xrightarrow{m_B} d, d \xrightarrow{j} c, d \xrightarrow{m_A} \text{err}$

k, c, d mean old-safe, inspection-clear and inspection-pending; c_0, c_1 are distinct controller states.

Component/controller alphabets are exactly their displayed labels; unlisted edges are absent. Unlisted non-error monitor edges self-loop, and err absorbs every label. Each comma-separated label denotes an edge.

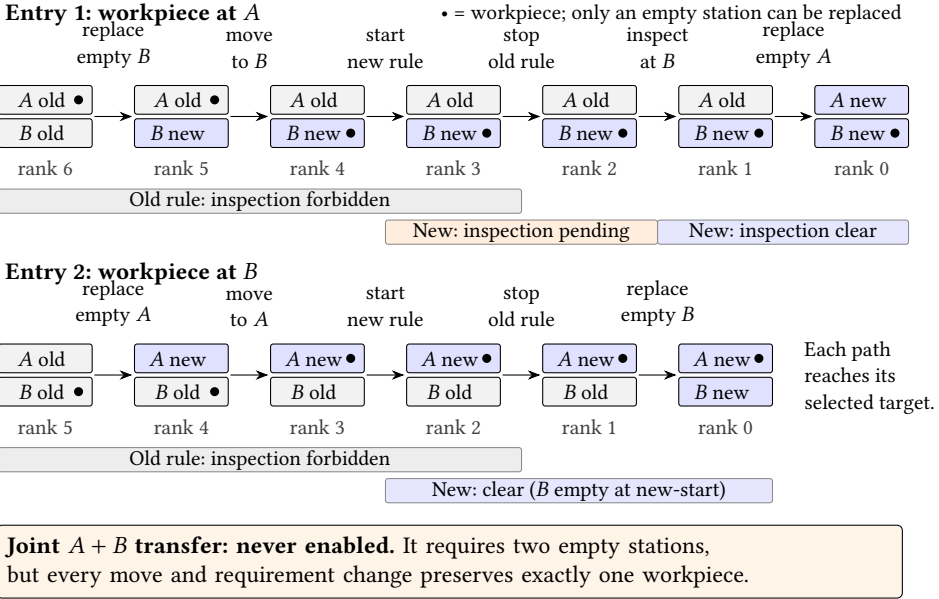


Fig. 1. A complete local policy from both old entries. Bands mark active rules and new-rule inspection memory; new-start precedes old-stop. Each arrow has one action and one outcome, decreasing rank toward its selected new target. Joint transfer merges these local commands with endpoints and requirements unchanged; this is not a DUCS encoding.

arrow decreases rank from six or five to zero, excluding safe but endless inspection. A joint transfer always requires both stations empty and is impossible. Technical Appendix B and S1 give the paths and closure.

2.2 Policy: Scheduling Requirement Lifetimes

Policy(2) makes the second choice explicit. Four bits record active old/new audit rules a_o, a_n and roles r_o, r_n . The interval invariant

$$(a_o \vee a_n) \wedge \neg(r_o \wedge r_n)$$

requires new-audit start before old-audit stop, but old-role stop before new-role start. The new audit monitor starts unlogged and requires a log before service. Each endpoint alternates its own log and service events and permits only its version's access role. With an identity component transfer, a completing policy clears any pending old service, starts new audit, stops old audit and role, transfers, and starts the new role. It reaches a compatible new endpoint from both old states; all outcomes decrease rank from at most six.

If the starts are merged and the stops are merged, the first start overlaps roles and the first stop removes audit coverage. Ordinary events and transfer preserve the four activity bits, so neither old entry has a completing merged policy. Technical Appendix C gives the complete finite input, policy and losing closure. This isolates independently scheduled requirement lifetimes.

3 Defining an Update Contract

The input supplies event control, fixed endpoints, components/transfers, old/new/interval requirements, initializers, optional precedence and handover targets, without prescribing a sequence. Section 4.1 connects initializers to requirement scopes.

3.1 Supplied Choices

Finite LTSs and safety monitors. A labeled transition system (LTS) $\mathcal{L} = (S, \Sigma_{\mathcal{L}}, \delta, s^0)$ has nondeterministic transitions $\delta : S \times \Sigma_{\mathcal{L}} \rightarrow 2^S$. Partition events as $\Sigma = \Sigma_{uc} \uplus \Sigma_c$: controllers may disable controllables but cannot select outcomes or suppress uncontrollable (UC) events. A synchronous-product event requires a transition in every component whose alphabet contains it; other components retain their state [23]. A monitor is an automaton that checks a requirement as events occur. Requirement r has a total deterministic monitor $t_r = (M_r, \Sigma, \eta_r, m_r^0, F_r)$ with absorbing errors for violating prefixes [1, 20]. The empty monitor product has one non-error state. Monitors observe labels; policies also observe successor component states.

Components and transfers. For each $i \in I$, let $e_i^{\text{old}}, e_i^{\text{new}}$ be the pre-update and post-update components and let $g_i \subseteq S_{e_i^{\text{old}}} \times S_{e_i^{\text{new}}}$ be their state-transfer relation. Define $g_i(s) = \{s' \mid (s, s') \in g_i\}$. The supplied g_i includes every modeled transformation outcome. Let $E^v = \parallel_{i \in I} e_i^v$.

Requirements and activation. Let pairwise-disjoint $R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$ contain the pre-update, post-update, and update-interval safety requirements, respectively. For each $r \in R^{\text{new}} \cup R^{\text{upd}}$, the input supplies a partial *monitor-initialization map* ι_r from version-tagged component tuples to $M_r \setminus F_r$. Its domain specifies where activation is allowed; in the cell, initialization records whether inspection is pending. Section 4.1 defines history-based and activation-scoped interpretations. For $r \in R^{\text{upd}}$, $\text{dom } \iota_r$ includes every reachable old component state.

Endpoints and handover targets. For $v \in \{\text{old}, \text{new}\}$, the supplied finite controller C^v preserves uncontrollables and makes $E^v \parallel C^v$ deadlock-free and satisfy R^v . Hiding controller auxiliary events leaves observation alphabet $\bigcup_i \Sigma_{e_i^v}$; the union over both versions is the ordinary alphabet Σ_N . Write Reach for initial-state reachability and

$$\text{CL}^v = E^v \parallel C^v \parallel \parallel_{r \in R^v} t_r.$$

Monitors are passive: every endpoint transition originates in $E^v \parallel C^v$; a monitor only updates on that label and adds no enabled event. The input supplies nonempty, checked targets $Z_{\text{load}} \subseteq \text{Reach}(\text{CL}^{\text{new}})$ with component, controller and new-monitor states. Cell uses a proper subset; the adapted benchmarks admit all reachable new-endpoint states. The supplied controller remains fixed.

Update commands and order. The set of update events, each executed exactly once, is

$$\Sigma_{\text{upd}} = \{\rho_i \mid i \in I\} \cup \{\text{stop}(r) \mid r \in R^{\text{old}}\} \cup \{\text{start}(r) \mid r \in R^{\text{new}}\}. \quad (1)$$

These controllable events transfer components, stop old monitors or start new ones, respectively, and are disjoint from ordinary events Σ_N : $\Sigma = \Sigma_N \uplus \Sigma_{\text{upd}}$ and $\Sigma_{\text{upd}} \subseteq \Sigma_c$. Each transfer event ρ_i is one atomic model transition: the controller chooses the update, but cannot choose its g_i outcome. They require uncontrollable quiescence: no modeled uncontrollable event may be enabled when an update command is issued.

An optional strict partial order $< \subseteq \Sigma_{\text{upd}} \times \Sigma_{\text{upd}}$ expresses precedence among update operations. For a pending-event set P , event a is order-enabled exactly when $a \in P$ and every $a' < a$ has left P . Monitors self-loop on update events that they do not specify.

Eligible contracts. An eligible contract satisfies the finiteness, typing, totality and non-error conditions above, and its dependencies form a strict partial order. Endpoint controller alphabets cover environment labels. Justifying initializers against intended requirement meanings is separate from eligibility. Interval and post-handover duties belong to R^{upd} and R^{new} ; $\text{start}(r^{\text{new}}) < \text{stop}(r^{\text{old}})$ enforces overlap.

3.2 States, Transitions, and Completion

States and all entry states. An update state $q = (\xi, \mu, P)$ records each component's version/state $\xi(i) = (v, s)$ with $s \in S_{e_i^v}$, each monitor's state $\mu(t_r) \in M_r \cup \{\perp\}$ ($\perp$ means inactive), and pending commands $P \subseteq \Sigma_{\text{upd}}$. Here $\rho_i \in P$ exactly when component i is old. Old monitors remain active until their stop leaves P ; new monitors become active when their start leaves P ; update monitors are always active.

Admission $\text{Init}(z)$ copies z 's component and old-monitor states. It initializes interval monitors through $\iota_r(\xi)$, leaves new monitors inactive and sets $P = \Sigma_{\text{upd}}$. The policy replaces C^{old} , discarding its state and scheduling restrictions; undeclared objectives, including old liveness, are not preserved. The entry set is

$$Q_0 = \{\text{Init}(z) \mid z \in \text{Reach}(\text{CL}^{\text{old}})\}. \quad (2)$$

All-entry coverage avoids waiting for a favorable old state; an admission predicate would define a different contract. In the cell, $Q_0 = \{q_A^0, q_B^0\}$ distinguishes the two holders.

Four kinds of induced transitions. For $\xi(i) = (v, s)$, component i uses e_i^v . The active-component product determines enabled ordinary uncontrollables $U(q)$, including those with unsafe outcomes, independently of updates and goals. For every $a \in \Sigma_{\text{upd}}$, impose

$$U(q) \neq \emptyset \implies \text{Post}(q, a) = \emptyset. \quad (3)$$

This global *uncontrollable quiescence* differs from transaction quiescence [19] and tranquility [38]. It is a command restriction, not a consequence of local transfer. Ordinary controllables may coexist with uncontrollables; the UC-priority lemma later justifies suppressing them. $\text{Post}(q, a)$ contains exactly the following outcomes; unmentioned coordinates remain unchanged. “Advance” means $\mu'(t_r) = \eta_r(\mu(t_r), a)$ for the specified active monitors.

Ordinary event $a \in \Sigma_N$. If a belongs to the local event set of at least one active component and every active component having a can transition, take every synchronous-product outcome and advance every active monitor.

Component update ρ_i . If $U(q) = \emptyset$, order-enabled, and $\xi(i) = (\text{old}, s)$, set $\xi'(i) = (\text{new}, s')$ for each $s' \in g_i(s)$, delete ρ_i from P , and advance every active monitor.

Requirement deactivation $\text{stop}(r)$. If $U(q) = \emptyset$ and order-enabled, advance every active monitor except t_r , retain inactive monitors at $\perp$, set $\mu'(t_r) = \perp$, and delete $\text{stop}(r)$ from P ; t_r does not observe it.

Requirement activation $\text{start}(r)$. If $U(q) = \emptyset$, order-enabled, and $\xi \in \text{dom } t_r$, advance every active monitor, retain other inactive monitors at $\perp$, install $\mu'(t_r) = t_r(\xi)$, and delete $\text{start}(r)$ from P ; t_r begins online observation afterward.

If a precondition is false, the successor set is empty; every nondeterministic outcome is included and environment-chosen. Let $\mathcal{Q}$ contain every state reachable from Q_0 through these transitions. *Safety and handover-ready goals*. An update state $q = (\xi, \mu, P)$ is safe, written $\text{Safe}(q)$, exactly when every active monitor is outside its error states. For $P = \emptyset$ and $z \in Z_{\text{load}}$, write $q \sim z$ when component states (without version tags) and all new-monitor states agree. The handover map κ selects such a full endpoint state z . Goals are

$$Q_{\text{goal}} = \{q = (\xi, \mu, \emptyset) \in \mathcal{Q} \mid \text{Safe}(q) \wedge U(q) = \emptyset \wedge \exists z \in Z_{\text{load}} : q \sim z\}. \quad (4)$$

At a goal, all components and endpoint monitors are new; interval obligations end and matching new monitors continue. Handover disables ordinary controllables and installs $\kappa(q)$'s controller state, outside Σ and the rank. No uncontrollable can compete. A matching UC-enabled state is not a goal; compatible targets may differ in later safe behavior.

State-dependent policies and finite completion. A memoryless update policy is a function $\sigma : \mathcal{Q} \setminus Q_{\text{goal}} \rightarrow 2^{\Sigma_c}$ selecting only events with nonempty successor sets.

The possible next states under this policy are

$$\text{Succ}_{\sigma}(q) = \bigcup_{u \in U(q)} \text{Post}(q, u) \cup \bigcup_{a \in \sigma(q)} \text{Post}(q, a). \quad (5)$$

The environment chooses any uncontrollable and any permitted-event outcome. An uncontrollable self-loop blocks updates by Equation (3) and permits a losing infinite run. The returned policy retains one controllable at a quiescent state.

Let $\text{Reach}_{\sigma}(Q_0)$ be the policy closure from Q_0 , stopping at goals.

Runs, winning, and witnesses. A run follows the policy successors and stops at a goal. It is *maximal* if infinite or ending at a goal or a state without successors. A single policy σ is winning for an FG-DUCS instance if, from every $q_0 \in Q_0$, (i) every reachable update state is safe, (ii) every reachable non-goal state has a successor, and (iii) every maximal update run reaches Q_{goal} after finitely many events. This *finite completion* counts observed transitions and assumes no fairness. A winning policy must disable a controllable self-loop during the update: allowing it would admit an infinite non-completing run. An uncontrollable self-loop makes its reached state losing. Ordinary actions can still make progress, as Cell's moves and inspection do; after handover the supplied new controller resumes its full matched behavior.

A *progress-ranking function* for a winning policy is $\text{rk} : \text{Reach}_{\sigma}(Q_0) \rightarrow \mathbb{N}$, with value 0 at goal states and satisfying $\text{rk}(q') < \text{rk}(q)$ for every non-goal state q and every $q' \in \text{Succ}_{\sigma}(q)$.

The FG-DUCS problem. Decide whether one policy safely completes the eligible game from every entry under every permitted event and outcome. Return $(\sigma, \text{rk}, \kappa)$ covering all Q_0 , or a losing-region certificate.

3.3 Why the Generated State Represents the Contract

A local update history $\ell = z(a_1, o_1) \cdots (a_n, o_n)$ records the old entry, events and realized local successors. Components follow their active-version transitions and transfers; monitors fold their declared active intervals. Commands obey precedence, guards and UC quiescence. A source policy selects controllable labels, retains every UC and permitted outcome, and safely completes every maximal history at a matching local quiescent target. Technical Appendix A gives these source operations independently of the game.

PROPOSITION 3.1 (ADEQUACY OF THE GENERATED UPDATE GAME). *For an eligible contract, map each local update history to its current component tuple, active monitor memories and unexecuted commands. Histories with the same image admit the same next events and all the same local outcomes, agree on safety, and have the same handover targets. This map gives a label- and outcome-preserving correspondence between local update histories and paths of the generated game. A history-dependent policy satisfying the source obligations exists exactly when a winning policy exists in that game.*

PROOF. Fold a source history into its current tuple $F(h)$. At entry this is $\text{Init}(z)$. Each ordinary step uses the same participating component edges and full Cartesian product of outcomes; each transfer uses the same quiescence/order tests and g_i , advances active monitors and consumes its command. Stop removes its own monitor before observation; start installs its initializer after observation by existing monitors. Thus every label and outcome has exactly the same successor image, and safety and matching targets agree.

Induction maps source histories to game paths. Conversely, choose an old tuple witnessing the entry and lift each edge by this one-step correspondence, also for infinite paths. A game policy lifts through F ; applying a source policy to consistently lifted game histories gives a history-dependent game policy. Both directions preserve UC events, all selected-label outcomes, unsafe prefixes, deadlocks and infinite avoiding runs. Theorem 5.1 supplies a state-based winner or excludes every history-dependent winner, completing the equivalence. $\square$

This proves correspondence for the supplied contract; requirement meaning and Java frontend correctness remain separate obligations. The construction reconnects each goal to the supplied endpoint.

PROPOSITION 3.2 (PRESERVATION OF THE SELECTED ENDPOINT). *For a returned WIN certificate and reached goal q , the new-phase submodel reachable after handover at q is label-preserving isomorphic to CL^{new} rooted at $\kappa(q)$. Hence their finite and infinite ordinary-event traces coincide.*

PROOF. Handover retains matching components and monitors and installs the selected controller state. Each new-phase state copies an endpoint state with exactly its labeled transitions, without further policy restriction or return to updating. This bijection preserves and reflects edges, hence finite and infinite paths. $\square$

Continuation properties established at the selected state are preserved; target selection can change which continuation applies.

Why matching and reconnection both matter. Cell's prefix $\rho_B, m_B, \text{start}, \text{stop}, \rho_A$ finishes every command with inspection pending. Physical matching alone admits clear target z_B , whose m_A violates that duty; monitor matching excludes the reset. Conversely, disabling controllable loop b in a one-state endpoint with loops a, b and goal *true* preserves legality, deadlock freedom and safety [31, Defs. 3.9–3.10, 4.3], but loses b -continuations. Reconnection preserves them.

Technical Appendix G characterizes policy reuse under changed endpoints, with a saved PC2 specialization.

3.4 What the Local Interface Generates

Table 2 compares native input and generation responsibilities, not general expressive power. A generated new controller can subsequently be fixed and reused, as in GR(1) updating. DUCS already constructs an update environment and goal; GR(1) updating already constructs a bridge. From the supplied local contract, FG-DUCS constructs mixed-component transitions, implements individual monitor lifetimes, and derives handover target tests.

Table 2. Input and generation responsibilities. Existing guarantees are retained in the comparison; the rows do not claim a general translation or measured modeling-effort reduction.

Interface	Supplied structure	Generated result and guarantee
DUCS [31, Defs. 4.1–4.3, 5.1]	Environments/goals, old controller, global transfer, transition requirements and update fluents	Update environment/goal, update-and-subsequent controller and controller-transfer relation; legal, deadlock-free completion from every old-controller state
GR(1) update [2, Def. 3.1, Secs. 4–4.1]	Old/new specifications and switching condition	New controller, bridge and switching bookkeeping; bounded switching and new-specification satisfaction from winning states
FG-DUCS	Local components/transfers, requirement lifetimes/initializers, supplied old/new controllers and loadable targets	Mixed game and WIN/LOSS certificates; local-history correspondence and selected-endpoint continuation

Concrete Cell constructions. Our DUCS witness supplies eight mixed stages, three endpoint states, empty-station guards, inspection/lifetime fluents and a target gate; DUCS generates its protocol. The GR(1) witness supplies version/lifetime variables, $p' = [h = B]$ at start, release at stop and a target test; its algorithm generates switching bookkeeping. Both admit the Cell paths; the appendices distinguish their endpoint guarantees. Technical Appendices H and I give the full analytical constructions, including the DUCS eventual-handover condition beyond native safety- T [31, Sec. 5.3]. These are neither solver runs nor general translations or encoding-size lower bounds.

4 What a Winning Policy Guarantees

A contract gives each requirement an active interval and an initial obligation. A winning policy enforces the supplied monitors over those intervals. Applying this result to a source requirement or an implementation also needs the connections in Table 3.

Table 3. From game guarantees to requirement meaning and execution.

Connection	Established result	Premises beyond the game
Game	Safe finite completion from every entry; matching target	Supplied successors and endpoint predicates (Theorem 5.1)
Requirement	Installed monitors enforce their continuation languages	Correct initializer key, observations and intended scope; checks below and in Technical Appendix E.1
Execution	Handover within the entry’s event rank	A1–A4 below; platform conformance and elapsed-time bounds remain unvalidated

4.1 Giving Requirements Their Intended Lifetimes

An initializer assigns the obligation present at activation; selecting any non-error state is insufficient.

An exact activation obligation. Cell’s owed-inspection bit starts at 1 exactly when B holds the workpiece. Event m_B sets it, j clears it, and m_A is forbidden while it is 1; other labels preserve it. Encoding 0 by c and 1 by d makes every monitor transition identical to this rule, including rejection of forbidden departure. Initialization and suffix induction prove exact enforcement. Figure 1’s first path starts with debt and clears it by ordinary inspection; the second starts clear.

Invariants that begin at activation. Activation-scoped interpretation ACT checks a pure invariant $\Box\varphi$ at the actual post-start observer vector and after every later label. It retains factual fluent values, not a past-violation bit. For deterministic observers with total update $d : V \times \Sigma \rightarrow V$, define

$$\text{ACT}_\varphi(v) = \{w \mid \forall u \leq w : \varphi(d^*(v, u))\},$$

where prefixes include the empty word. A faithful safety monitor M has non-error language $S_\varphi = \text{ACT}_\varphi(v_0)$ from its initial state. Let $L_M(m)$ denote its continuations from state m .

PROPOSITION 4.1 (SAFE LOOKUP CELLS FOR PURE INVARIANTS). *If a lookup at v selects a non-error state m reached together with v by a safe word in the unrestricted monitor–observer product, or a state with the same continuation language, then $\varphi(v)$ holds and $L_M(m) = \text{ACT}_\varphi(v)$.*

PROOF. Let h reach the pair (m_h, v) . Faithfulness and non-error reachability give $h \in S_\varphi$, hence $\varphi(v)$. For every suffix w ,

$$w \in L_M(m_h) \iff hw \in S_\varphi \iff w \in \text{ACT}_\varphi(v).$$

The second equivalence uses safety of the prefix h and deterministic observers: future valuations depend only on v and w . Language equivalence transfers the equality to m . $\square$

The witness need not be plant-executable or obey one-shot update restrictions. This is residual-language reasoning [12, 33], applied to the generated lookup. At activation, the adapter must choose the actual post-start key and use the same future observation rules; start is consumed once. Undefined entries and fallback initialization are outside the criterion. A true $\varphi(v)$ alone does not supply a lookup entry.

Independent source and table checks establish the criterion for 900 non-error cells in 138 saved tables (Technical Appendix E.1). Applying it to a particular policy still requires those key and observation premises. PC2 and GSM below have direct arguments on reached tuples; the table checks do not validate all benchmark activations.

When prior histories are part of the requirement. For prefix-closed requirement language S_r , declared activation histories $H_r(\xi)$ and monitor continuations $L_r(m)$, residual soundness (RS) means $L_r(l_r(\xi)) \subseteq \bigcap_{h \in H_r(\xi)} \{w : hw \in S_r\}$. New histories include start once; monitors consume later events. Equality is exact; strict inclusion can reject feasible updates, and an empty history set makes inclusion vacuous. Scope completeness and intent are external obligations (Technical Appendix B.1).

A *fluent* records setting/resetting events. The saved checks use **A** for NEW (safe pre-activation histories consistent with the component observation) and **E** for UPD (reference beginning at entry). **H**, which counts prior violations too, is not claimed (S3). Cell’s pending initializer is exact for occupancy-based debt but conservative for a start-only reference that starts clear. An **A** inclusion alone establishes neither the occupancy duty nor ACT.

4.2 Execution and Handover

The execution connection has four distinct premises (Technical Appendix F):

A1: endpoint observations preserve component/monitor states and same-labeled endpoint edges.

A2: update control disables only unselected controllables, preserves every UC event/outcome, reports the successor and eventually observes a next event.

A3: commands start quiescently and finish atomically, finitely and failure-free with an observed modeled successor.

A4: entry observes/copies the actual old state; quiescent handover installs the selected compatible state, using finite failure-free linearizable boundaries [16].

Under A1–A4, every execution maps to the linked model and reaches handover within the entry rank (Theorem F.1). History completeness means that every actual activation history, with its declared reference origin, belongs to the supplied history scope; with RS it justifies the residual interpretation. No event-choice fairness is assumed. Internal divergence and unobservable outcomes are excluded; rank bounds events, not time.

5 Synthesizing and Checking an Update Policy

The procedure specializes strong-planning and on-the-fly reachability principles [7, 8, 37], retaining every queried event outcome and producing evidence for every old entry.

A *bucket* is $\text{Post}(q, a)$; empty means disabled. Disabling controls at a UC-enabled state removes runs without creating deadlock, preserving winning (*UC priority*, Lemma A.1). The proof below also establishes memoryless sufficiency using ranks and a history-independent losing-region response.

5.1 Lazy Successor Generation

Algorithm 1 constructs a reachability attractor [5]. V contains discovered states, X those with all UC candidates queried, and W proved winners. A UC state needs all successors to win; a quiescent state needs one complete control bucket. Lazy queries controls incrementally, keeping unfinished cursors in Retry. Each promotion records a complete nonempty proving set and a rank above its members. Reverse incidences propagate promotions. Failure requires exhausted candidates and a promotion fixed point. Extraction chooses lower-rank controls and compatible targets; it need not minimize path length.

5.2 Correctness and Certificates

THEOREM 5.1 (TERMINATION, SOUNDNESS, AND COMPLETENESS). *For every eligible finite FG-DUCS instance with Equation (3) and quiescent goals in Equation (4), Algorithm 1 terminates. A returned $(\sigma, \text{rk}, \kappa)$ keeps every active monitor non-error, preserves every uncontrollable event and every outcome of each policy event, reaches Q_{goal} finitely without deadlock, and gives each reached goal a compatible target in Z_{load} for the supplied game. It returns no solution iff no winning policy covers Q_0 in that supplied game.*

PROOF. *Coverage.* Every new safe non-goal enters Open and leaves only after expansion queries all UC candidates. At quiescence, finite control cursors retain unfinished nonwinners in Retry; promotion removes only winners. Thus expanded states have complete UC outcomes, unfinished nonwinners remain retryable, and unexpanded safe non-goals remain open.

Termination. Each state is expanded/promoted once and each candidate queried once. Reverse incidences are processed on promotion, so the finite search terminates.

Soundness. Goals start at rank zero with compatible handover states. By induction, each promotion adds a safe state above a complete nonempty proving set of previously ranked winners: every

Algorithm 1: Lazy**Input:** Typed update contract (Section 3)**Output:** (σ, rk, κ) or (no solution, L)

```

1  $V \leftarrow Q_0$ ;  $X, \text{Retry} \leftarrow \emptyset$ ;  $W \leftarrow V \cap Q_{\text{goal}}$ ;
2  $\text{Open} \leftarrow \{q \in Q_0 \mid \text{Safe}(q) \wedge q \notin Q_{\text{goal}}\}$ ; set  $rk(q) \leftarrow 0$  for  $q \in W$ ;
3 while true do
4   Add queued goals to  $W$  once with rank zero; exhaust Promote, retaining witnesses/ranks, removing winners
   from Retry and propagating predecessors;
5   if  $Q_0 \subseteq W$  then return  $(\sigma, rk|_{\text{Reach}_\sigma(Q_0)}, \kappa)$  constructed from the goal witnesses and retained lower-rank
   choices;
6   if  $\text{Open} \neq \emptyset$  then
7     Remove  $q$  from Open, add it to  $X$ , and query all uncontrollable candidates;
8     Close the control cursor if UC-enabled; otherwise initialize it and query to the next nonempty bucket or
     exhaustion;
9     Insert  $q$  into Retry iff  $q \notin W$  and its cursor has remaining candidates;
10  else
11    if  $\text{Retry} = \emptyset$  then return (no solution,  $L \leftarrow V \setminus W$ );
12    Remove  $q$  from Retry; query to the next nonempty bucket or exhaustion; reinsert iff  $q \notin W$  and its
    cursor has remaining candidates;
13  Add every queried successor to  $V$ ; queue newly discovered goals for rank zero and safe non-goals for Open
    (unsafe states stay outside  $W$  and Open);

```

UC successor, or every outcome of one control event. Queried buckets are immutable. Extraction retains all UC events, chooses one such control bucket at quiescence, and disables other controls. Every reached non-goal is safe, has a successor, and every successor has strictly smaller natural-number rank. Consequently no maximal run is an infinite run or a non-goal deadlock, and every run reaches a matching quiescent goal. The monitor rules advance exactly the memories active over each event; induction from entry establishes every declared safety interval. Handover retains the matching new memories.

Completeness. At failure both exploration queues are empty and promotion has reached a fixed point. Some entry is in $L = V \setminus W$, and L contains no goal. Coverage ensures that each safe state in L was expanded. If it is UC-enabled, at least one UC outcome lies in L , since otherwise it would be promoted. At quiescence every control candidate is exhausted; each enabled control bucket has an outcome in L for the same reason. Against any history-dependent policy the environment can select such an outcome after every history. Disabling all events deadlocks; otherwise the resulting run reaches an unsafe state or stays outside the goals forever. Hence no winning policy covers that entry. A winning policy therefore rules out failure and, by termination, forces success. Finally, UC priority loses no winning policy: removing optional controls at a UC-enabled state removes runs without introducing a deadlock, and the restricted policy remains admissible in the original game. $\square$

Checking the returned evidence. WIN checking requires exactly $D_\sigma = \text{Reach}_\sigma(Q_0)$ as the rank domain, safety, enabled selected controls, nonempty all-outcome closure and strict decrease at non-goals. Goals require rank zero, quiescence and a compatible κ , defined exactly on $D_\sigma \cap Q_{\text{goal}}$; omitted policy entries are empty. LOSS checking requires an entry-containing, goal-free L with the UC/all-control counter-outcomes used in the completeness proof. These certify search relative to the successor and endpoint predicates [18]; Section 6 states their implementation trust boundary.

5.3 Exploring the Compiled Contract on Demand

With indexed predicates and constant-time bookkeeping, search is linear in discovered states, queried candidates and returned outcomes, excluding preprocessing and Java sorting/scans. Lazy may explore the whole reachable graph. For independent one-state updates, transfer-first Lazy instead visits $n + 1$ states versus Direct-Full’s 2^n subsets; Technical Appendices K and O give the accounting and complete default-order proof.

6 Implementation and Evaluation

6.1 Design and Implementation

RQ1 tests feasibility mechanisms on constructed fine/merged pairs. RQ2 separates finite reference games, application certificates and source-monitor checks. RQ3 compares exploration and cost on 27 adapted contracts. Extended budgets and auxiliary variants remain separate from fixed-budget medians.

The 9 inherited inputs each have three variants: Base has no interval requirement, R1 restricts ordinary actions, and R2 relates boundaries to transfers. PC2-Rolling is a separate source-derived availability case. Legacy synthesizes subsequent control as well as updating, a different objective.

Implementation. Our Modal Transition System Analyser (MTSA) extension [10] checks certificates [27]; Link checks old–update–new composition, entries and targets. An independent oracle parses restricted Finite State Processes (FSP). Finite-JSON runs save losing membership; FSP benchmark runs report only its statistics.

Use of generative AI. Generative AI assisted contract and example design, proof drafting, Java solver/frontend implementation, Python checkers, experiment drivers, execution, result analysis and manuscript editing. Validation used certificate and exhaustive checks for algorithms, source and automaton comparisons for initialization, saved-output checks for tables and primary sources for citations. Claude Opus 5.5 and ChatGPT Pro/Latest supplied editorial reviews, not empirical evidence. Authors retain responsibility; shared AI/code can produce correlated faults. Technical Appendix P details tool roles; no human evaluation was performed.

Constructing the benchmark inputs. Nine inputs adapt eight DUCS sources; PC1/PC2 share a parent. Adaptations make PC tool results UC, shorten Railcab transfers preserving both outcomes, omit PowerPlant’s initial ENV pair, and change safety/ordering; equivalence is unestablished. Technical Appendix O specifies every adaptation.

FSP goals and component/transfer declarations feed MTSA’s GR endpoint backend. Targets are all reachable new-endpoint states. The frontend compiles monitors, enumerates physical closure and fills global-tuple initializer tables from current observers; interval tables use post-entry values. Observers track nonlocal labels and survive commands/transfers. Lazy generates the remaining product on demand; JVM cost includes preprocessing.

Inputs have 1–5 components, 4–131 entries and 3–26 commands (Technical Appendix L). Only Railcab transfers branch; none declares precedence. R2 encodes order in monitors. GSM/PC1’s single transfer makes merging the identity.

Exploration variants. All methods share entries, successors, goals, checks and terminals. The on-the-fly (OTF) variants omit controls at UC-enabled states and stop when all entries win. Direct-Full first materializes reachable buckets including those controls, then solves with the same UC-priority rule. Full+UC is an auxiliary full-construction variant that omits those controls during construction. At quiescence Lazy queries individual buckets, Eager all controls, and Update-first favors updates, testing O-DUCS’s priority [17] within our solver. Lazy favors stops/transfers, prioritizes

states by endpoint mismatch and pending-command counts, and uses LIFO exploration before FIFO retries (Technical Appendix K); this order need not minimize exploration.

Design and measures. RQ3 uses one Windows/Xeon host, 64 GiB heap and 1,200 s JVM cap: 27×4 same-game cells plus 27 Legacy references. Medians require five completions; censored cells retain their capped trial. Solver time includes checking; total JVM adds frontend, startup, output and endpoints. We also report exploration, policy size and peak working set. Legacy uses its native timer; Mac pilots are excluded. Full settings and interruptions remain in Technical Appendices P and O.8 and S4.

6.2 RQ1: Can Local Choices Change Feasibility?

Cell and Policy isolate component and requirement-lifetime obstructions (Section 2). These new cases exercise choices absent from the inherited DUCS-oriented benchmarks. We test whether merging those choices, with the remaining contract fixed, removes all completing policies; this is a mechanism question, not a frequency estimate over deployments.

What is merged. Partition transfers, old stops and new starts separately into blocks. A transfer block $B \subseteq I$ simultaneously permits all targets in $\prod_{i \in B} g_i(s_i)$; a boundary block removes or initializes its monitors at the same component tuple. Other active monitors compose member-label effects, checked to commute pairwise on every monitor/residual state. Mapped precedence must remain irreflexive and acyclic. Ordinary behavior, entries, endpoints, relations, initializers and targets stay fixed; quiescence and all member guards apply. These are comparison contracts, not DUCS encodings.

The following obstruction explains the Cell, Policy and PC2 merges directly; the rolling threshold then gives an exact quantitative boundary.

LEMMA 6.1 (MANDATORY-COMMAND OBSTRUCTION). *Let q_0 be an entry and B a nonempty set of commands pending there. Suppose a safe region R contains q_0 , retains all commands in B , and is closed under every safe successor whose label is outside B . If each command in B is disabled or has an unsafe outcome at every state in R , no winning policy covers q_0 .*

PROOF. Every goal exhausts pending commands, so a winner must execute a first command in B . Safety and closure keep its prefix in R , where that step is disabled or has an environment-chosen unsafe outcome. Avoiding it yields a non-goal deadlock or infinite run, also losing. $\square$

In Cell’s one-workpiece region, joint transfer is disabled. Policy retains its initial active bits until either grouped boundary violates the interval. PC2-Rolling’s enabled joint transfer violates availability (Section 6.2.1). These results concern specified merges, not native DUCS expressiveness.

Mechanisms and scope. Readiness/rejoining gates motivate staged replacement [29, 35]. Rolling begins with n ready replicas and no spares; its interval requirement keeps at least m ready. Each transfer boots its replicas; each has one UC readiness event and no waiting loop. Service self-loops are controllable. The target is all-new and ready.

PROPOSITION 6.2 (ROLLING THRESHOLD). *For $n \geq 2$, $1 \leq m < n$ and $1 \leq k \leq n$, fixed consecutive batches of size k (with a smaller final remainder) admit safe finite completion iff $k \leq n - m$.*

PROOF. Some mandatory batch has size k . Its transfer leaves at most $n - k$ ready replicas, so $k > n - m$ violates the interval. Otherwise, transfer one batch only when all replicas are ready, disable service, and retain every UC readiness outcome before the next transfer. Readiness never falls below $n - k \geq m$. With u old and b booting replicas, $2u + b$ strictly decreases on every selected

transfer and every UC event, reaching zero exactly at the target. Thus all maximal runs finish without fairness. $\square$

The 15 Rolling separations compare $k = 1$ with $k = n$; the 70 threshold cells are separate. Canary adds health reports/recovery, DB-Rolling role-sensitive replacement, and Rolling+Audit overlapping duties. Policy isolates opposing boundaries; Threads supplies both-WIN/both-LOSS quiescence controls. Of the 33 separations, 30 are parameter points in Rolling and Canary; they are not 33 independent mechanisms. Table 4 keeps all outcomes. Technical Appendix O.11 specifies mechanisms and assumption controls; their deployed prevalence is unestablished.

Answer. The suite yields 33/55 fine-WIN/merged-LOSS comparisons across five families. PC2 contributes no solver WIN/LOSS pair because its merged run times out; the source argument below independently proves merged impossibility.

Why finer is not always better. Two components with one state per version share UC labels x, y . The only UC loops are y in old 1, x in new 1, x in old 2 and y in new 2; shared labels synchronize. Thus all-old and all-new are quiescent, but either mixed product has a UC loop. With identity transfers and no requirements, either first local transfer prevents quiescence forever, so fine loses. A joint transfer skips the mixed state and wins at rank one. A shared controllable loop makes both endpoints deadlock-free. Atomicity can therefore remove either a necessary or a harmful intermediate state; there is no general refinement order (Technical Appendix O.12).

6.2.1 A Source-Derived Calibration Case. PC2-Rolling adds calibration while requiring another arm operational. Requests are controllable, completions UC, and transfers enter calibration while preserving initializer observers. Erasing calibration recovers the original operational process, without whole-problem equivalence. Endpoints retain baseline requirements plus availability; the interval additionally treats transfers as calibration entry. The 25 boundaries and two transfers have no supplied order (Technical Appendix J).

The saved policy calibrates arm 1, waits for completion, then calibrates arm 2 and waits again. All 126 entries are covered, every outcome decreases rank from at most 37, and at least 1 arm stays outside calibration. Link checks the full composition. Each of the thirteen source invariants holds at its start and through the remaining start-only suffix until handover, proving ACT on reached tuples. This covers neither all initializer tuples nor all 138 occurrences; GSM below exercises ordinary actions under an active new requirement.

History-wide obligations give a different result: 87/126 entries fail the safe-full-prefix check (21 before entry, 66 during update), and all admit an earlier violating cycle restoring zero observers. Safe-history nonemptiness for 38,288 initializer tuples and runtime remain unchecked. Technical Appendix J retains each source formula, activation path and history partition.

Why the fixed merge loses. Every old entry admits both raw transfers. Before joint transfer, old plants lack calibration actions and all other steps preserve readiness. The joint transfer therefore puts both arms in calibration, clearing both interval readiness bits and reaching error. This proves impossibility, while the merged solver and fine Direct-Full both time out at 7200 s/200 GiB. Those timeouts remain unresolved solver results.

Inherited inputs and additional witnesses. On nine inherited inputs, merging transfers, boundaries or both changes 0/54 Base/R1 decisions. Their total raw transfers omit Rolling’s startup phase; totality alone does not ensure safe merging. States decrease in 44 comparisons and stay equal in 10. R2 names individual events and is excluded. This null result limits observed granularity benefits.

Additional restricted-transfer comparisons retain both-LOSS and incomplete cases separately (Technical Appendix O; S1–S4).

Table 4. Granularity mechanisms, parameter sweeps and controls. Each pair compares fine and merged contracts of one model: Policy merges boundaries, other rows merge transfers. Counts are comparisons, not independent applications. The main total excludes Cell.

Family / control	Mechanism and parameters	Pairs	WIN / LOSS	LOSS / LOSS	WIN / WIN	Inc.
<i>Constructed families and parameter sweeps</i>						
Rolling	Readiness during startup. $n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Canary	Failure reports and recovery. $n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Policy	Audit overlap; role exclusion. 2 rule pairs	1	1	0	0	0
DB-Rolling	Role-gated replacement. $n = 3, m = 2, 3$	2	1	1	0	0
Rolling+Audit	Readiness and audit overlap. $n = 2, m = 1$; 1 audit pair	1	1	0	0	0
<i>Quiescence control</i>						
Threads (control)	Backpressure or persistent arrival. $n = 2 \dots 6$; queue 1,2	20	0	10	10	0
<i>Source-derived attempt</i>						
PC2-Rolling	Availability during calibration. 2 arms; floor 1	1	0	0	0	1
Main total		55	33	11	10	1
Cell _n (reference)	Empty-station transfer guards. $n = 2 \dots 10$	9	9	0	0	0

Outcomes read fine / merged; Inc. is unresolved. PC2’s merged solver remains TO, not LOSS. Rolling and Canary share a capacity obstruction: their sweeps supply 30 of the 33 WIN/LOSS pairs. n counts replicas/workers/stations. Canary bounds reported failures by $n - m$; otherwise m is an availability floor. Threads varies queue capacity 1,2 under both arrival regimes. Rolling+Audit’s boundary merge remains WIN and adds no pair here.

6.3 RQ2: Do Policies Satisfy the Contract?

Answer. On the finite validation fixtures, Lazy and Direct-Full match all expected results from independently reconstructed labeled transition systems (LTSs). Application certificates additionally check the shared Java game semantics; source-level initialization evidence is reported separately below.

Independent reconstruction and certificates. Python independently constructs graphs and certificates for 46 fixtures: $46 \times 2 = 92$ jobs yield 36 WIN, 48 LOSS and 8 invalid (missing events, deadlocks or suppressed UC). All 84 eligible jobs pass certificate/exhaustive checks; all WIN jobs pass Link. Explicit successors are separate; frontend and endpoint/goal code are shared. These jobs are not independent applications.

Six monitor-free fixtures independently reconstruct restricted FSP, endpoints, complete games and goals. All 138 NEW tables match 1,587 reconstructed entries. A separate source parser checks the 46 invariant positions across three variants: initial truth, source observer rules and all 35,283 transitions from 900 safe monitor–observer pairs agree. Induction gives source-invariant fidelity

for these saved objects and their checked alphabets, not arbitrary FSP compilation, physical decoding or actual activation conformance (Technical Appendices D and E.1).

What application WIN and LOSS mean. Railcab’s 22-entry Base policy permits 1–4 ordinary events between old-stop and new-start; R1’s saved policy has no ordinary gap events and retains seven interval guards. Both pass certificate/Link checks. A continuously needed duty must therefore be declared as an interval requirement; WIN alone does not supply it (Technical Appendix O.9).

Industry/R1 loses at 42/131 entries. In its final trap, GF1/TOR validation violates new monitors; DSD1 admits an endless rejection cycle, while withholding its only safe validation at quiescence deadlocks. Exhaustive closure excludes every order. Native DUCS wins on its different subsequent-control problem. Full case details and diagnostic variants are retained in Technical Appendix O and S4.

Checking initializer interpretation. There are 138 safe-history NEW inclusions and 135 entry-scoped UPD justifications (46 R1/89 R2), distinct from ACT. Empty scopes pass vacuously. Technical Appendix D gives all formula, label, reset and automaton checks.

Source obligation over ordinary actions. GSM changes $\square\text{-decode.3}$ to $\square\text{-decode.2}$. From every old entry, reach the first RECEIVED state; then

$$\text{RECEIVED} \xrightarrow{\text{stop,start}} \text{RECEIVED} \xrightarrow{\text{decode.3}} \text{DECODED} \xrightarrow{\text{out put}} \text{SENDER} \xrightarrow{\rho} \text{SENDER}_{\text{new}}.$$

The preceding *receive* sets the *decode.2* observer to zero; boundaries preserve it. The saved initializer maps zero to monitor state zero: *decode.2* enters error, all other labels preserve zero. Thus the new obligation constrains an actual ordinary choice. Transfer matches sender, observer and new-monitor zero to the endpoint. The pre-stop distance is at most seven; adding the five displayed events gives rank at most twelve. Consecutive stop/start satisfies R1’s gap ban. This source-derived analytical policy proves ACT, not a new solver result; a separate 45-pair source check establishes E initialization (Technical Appendix E).

The 25 WIN certificates have 8–1,071 update states and maximum ranks 4–34, consistent across five repeats. Linked old-update-new controllers have 15–1,260 states (S4). Ranks bound events, not downtime or optimal length (Technical Appendix O.8).

6.4 RQ3: Does Lazy Reduce Exploration?

Answer. Lazy avoids states and queries in all 14 completed Direct-Full pairs. Within the fixed budget it decides 26 contracts, Eager 17 and Direct-Full 14; all paired decisions agree (Table 5). Update-first is faster on 8/24 paired solver medians but adds two timeouts. Lazy returns LOSS for Industry/R1 and times out on PC2/R2 at 1,200 s/64 GiB (fixed), 3,600 s/64 GiB (sensitivity) and 7,200 s/200 GiB (S4, earlier series); the limiting resource is unknown.

Legacy yields 9 WIN, 13 OOM and 5 unmeasured snapshot-cap cases. Its differing subsequent-control objective precludes same-game ratios or impossibility inferences (Technical Appendix O).

Figure 2 plots all 14 completed pairs; Technical Appendix M gives exact states, queries and costs. In R2, Update-first uses fewer states and a lower solver median on six of eight completed pairs, showing order sensitivity (Technical Appendix O).

Capped Direct-Full attempts give total-JVM lower bounds, not solver ratios or timing medians (Technical Appendix O.10).

Table 5. All 27 contracts per method. TO denotes one capped trial, not a timing median. Ratios summarize heterogeneous completed pairs; values above one favor Lazy.

Method	WIN/LOSS/TO	Faster / pairs	Solver ratio	Total JVM ratio
Lazy	25/1/1	—	—	—
Eager	17/0/10	0 / 17	2.04 [1.18–4,575.05]	1.60 [1.00–144.85]
Update-first	23/1/3	8 / 24	1.07 [0.19–39.76]	1.00 [0.32–6.10]
Direct-Full	14/0/13 [†]	0 / 14	9.14 [1.58–2,123.81]	2.27 [1.00–130.66]

TO: timeout; no OOM/unmeasured cases. [†] Workflow/R2 retains its interruption and later same-setting timeout (S4). Comparator/Lazy ratios are median [min–max] across completed pairs of five-run medians. Faster / pairs counts lower comparator solver medians among all completed pairs. JVM includes frontend, startup, endpoints and output; solver includes checking. Unresolved costs have no ratio.

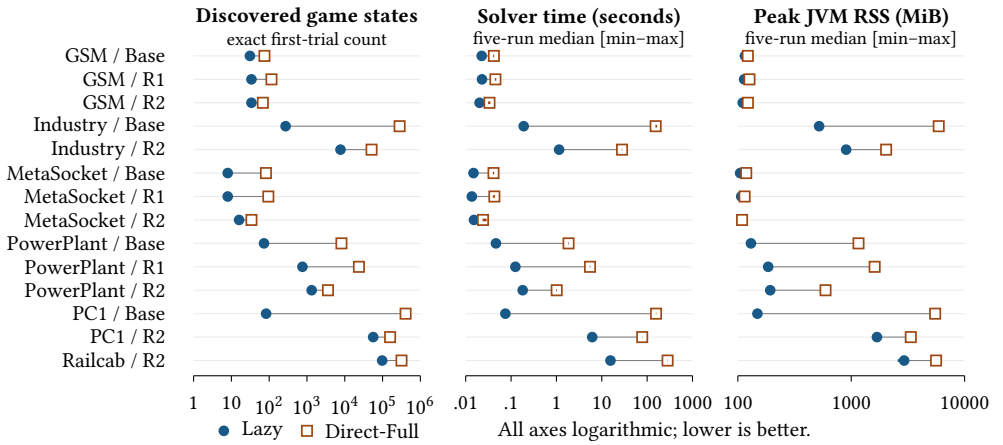


Fig. 2. All 14 completed WIN/WIN pairs. Solver time includes checking but excludes frontend, endpoint preparation and JVM startup. RSS is sampled resident memory over the whole JVM run, not solver-only or heap usage. The other 13 Direct-Full contracts time out (Figure A4) and are omitted. This compares whole procedures.

Separating construction choices. Lazy versus Eager isolates deferred controllable-bucket generation under the same UC pruning, input, host and budget: 26 versus 17 decisions. Direct-Full additionally changes pruning. Auxiliary Full+UC timings cannot isolate that effect across differing builds, hosts and repetitions; no unpruned OTF is implemented (Technical Appendix O.10).

Scaling limits. All methods finish the independent family for $n = 2, \dots, 20$. At $n = 20$, Lazy and Update-first discover 21 states each, Eager 211, and Direct-Full 1,048,576 (S4).

Travel reaches timeouts and out-of-memory failures under all methods, including expanded-budget timeouts. These separate trials never enter medians; Technical Appendix O.10 retains all populations and adverse outcomes.

7 Related Work

Updating and preserving existing control. DUCS and GR(1) updating generate the intervening control from different native inputs; Section 3.4 and Table 2 give concrete Cell constructions. O-DUCS separates update and post-update analysis [17, Sec. 3]. The fork’s plain OTF precomputes

a new controller and reconnects matching environment/safety states [4]; identity with the 2024 release is unestablished. FG adds supplied endpoints/targets, local transfers, individual lifetimes and initializers to this existing reconnection.

On-demand synthesis. DCS already explores component products on demand, defers controllable alternatives, treats mixed states as uncontrollable, and propagates success or failure backward [8, Sec. IV]. Its 2016 formulation addresses safety and reachability; the later OTF-DCS synthesizes safe nonblocking directors [9]. FG specializes these search patterns to its generated contract, with complete event-outcome buckets and all-entry certificates for requirement lifetimes and matched handover. Compositional and modular control instead structure specifications for synthesis [28, 39].

Requirements across adaptation. Temporal adaptation [40], scenario-based updatability [13, 25] and Live Synthesis [12] address obligations across change. Our initialization uses scoped requirements [11] and residual reasoning [33]. State attraction bounds startup under weak invariance [32]. Thuijsman and Reniers [36, Secs. 4–7] allow individual or simultaneous reconfiguration, requirements that depend on features, and state retention or reset. Their supervisor permits a marked continuation from every reachable state; FG requires every maximal update run to finish at a supplied controller.

Operational update mechanisms. Local updating [24], PROTEOS [14, Secs. 4.3–4.6], workflow updating [34, Secs. 4.3, 5.2], Kitsune [15, Sec. 2] and network synthesis [26] address replacement, migration and transition safety. Ma et al. enforce transaction version consistency using dynamic dependencies and sufficient freeness, permitting concurrent versions and selective blocking [22, Secs. 4.1–4.3]. These mechanisms differ from global UC quiescence; finite handover alone establishes no transaction consistency.

8 Threats to Validity

Permanently enabled UC arrivals or polling block even local commands. Threads contrasts finite UC phases with perpetual activity. Fairness-dependent waiting is outside the contract; inherited-adaptation effects on feasibility are not isolated. Decoding, history completeness and A1–A4 remain unvalidated; repair is excluded. Constructed cases assume finite startup, reporting, recovery and election; selection may favor separations, and deployed prevalence is unestablished.

Shared authorship, AI and code risk correlated faults; independent parsing covers restricted FSP. Direct-Full is in-house; Legacy’s objective differs. Pruning isolation, external solvers and randomized orders remain untested. Ratios omit unresolved costs; extended trials lack medians. Sampling can miss peaks; RSS identifies neither heap occupancy nor bottlenecks. S4 retains interruptions.

Null/LOSS results, faster Update-first, lower Eager/Update-first memory and Travel/PC limits bound gains. PC2’s merged timeout remains unresolved. Fixtures/repeats are not independent applications; timing uses one machine. Inherited inputs lack precedence; only Railcab transfers branch. Exponential separation assumes the default order.

9 Conclusion

FG-DUCS compiles local transfers, requirement lifetimes and monitor initialization into policies connecting verified controllers. Contract correspondence and certifying synthesis establish safe finite completion from every admitted entry and outcome, preserving the chosen new controller’s continuation. Complete examples explain granularity-dependent feasibility; experiments show reduced exploration and more decisions within a fixed budget. Guarantees remain relative to declared requirements, uncontrollable quiescence and faithful execution.

Data Availability

We created and publicly released the anonymous FG-DUCS replication package for this paper [4]. The retained evidence snapshot is available at <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-invariants-20261002>. The paper-matched release cited in [4], fgducs-c75-20261003, contains this revised main paper and paper/supplementary_material.pdf: Part I is Technical Appendix A–P; Part II is Supplement S1–S5. Its reading guide, bookmarks and continuous page numbers locate the proofs, full models and experimental records cited throughout the main text. The repository README links directly to both parts and maps paper sections to evidence. The package also provides source code, all 27 application contracts, scripts and saved results; saved-evidence checks require no new performance experiments. Dependencies come from official upstream sources.

References

- [1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. doi:10.1016/0020-0190(85)90056-0
- [2] Gal Amram, Shahar Maoz, Itai Segall, and Matan Yossef. 2022. Dynamic Update for Synthesized GR(1) Controllers. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 786–797. doi:10.1145/3510003.3510054
- [3] Jesper Andersson, Luciano Baresi, Nelly Bencomo, Rogério de Lemos, Alessandra Gorla, Paola Inverardi, and Thomas Vogel. 2013. Software Engineering Processes for Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II*, Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.). Lecture Notes in Computer Science, Vol. 7475. Springer, Berlin, Germany, 51–75. doi:10.1007/978-3-642-35813-5_3
- [4] Anonymous FG-DUCS Authors. 2026. FG-DUCS Evaluation Artifact. Public anonymous replication package prepared for this paper, including Technical Appendix A–P, Supplement S1–S5, source code and saved evidence; tag fgducs-c75-20261003. <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-c75-20261003>
- [5] Franck Cassez, Alexandre David, Emmanuel Fleury, Kim G. Larsen, and Didier Lime. 2005. Efficient On-the-Fly Algorithms for the Analysis of Timed Games. In *CONCUR 2005 – Concurrency Theory (Lecture Notes in Computer Science, Vol. 3653)*. 66–80. doi:10.1007/11539452_9
- [6] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cucik, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaella Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. 2009. Software Engineering for Self-Adaptive Systems: A Research Roadmap. In *Software Engineering for Self-Adaptive Systems*. Lecture Notes in Computer Science, Vol. 5525. Springer, Berlin, Germany, 1–26. doi:10.1007/978-3-642-02161-9_1
- [7] Alessandro Cimatti, Marco Roveri, and Paolo Traverso. 1998. Strong Planning in Non-Deterministic Domains via Model Checking. In *Proceedings of the Fourth International Conference on Artificial Intelligence Planning Systems (AIPS '98)*. AAAI Press, Menlo Park, CA, USA, 36–43. <https://cdn.aaai.org/AIPS/1998/AIPS98-005.pdf>
- [8] D. Ciolek, V. Braberman, N. D'Ippolito, and S. Uchitel. 2016. *Directed controller synthesis of discrete event systems*. Technical Report 1605.09772. arXiv.
- [9] Daniel Ciolek, Matias Duran, Florencia Zanollo, Nicolás Pazos, Julián Braier, Víctor Braberman, Nicolás D'Ippolito, and Sebastián Uchitel. 2023. On-the-Fly Informed Search of Non-Blocking Directed Controllers. *Automatica* 147 (2023), 110731. doi:10.1016/j.automatica.2022.110731
- [10] Nicolas D'Ippolito, Dario Fischbein, Marsha Chechik, and Sebastian Uchitel. 2008. MTSA: The Modal Transition System Analyser. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 475–476. doi:10.1109/ASE.2008.78
- [11] Matthew B. Dwyer, George S. Avrunin, and James C. Corbett. 1999. Patterns in Property Specifications for Finite-State Verification. In *Proceedings of the 21st International Conference on Software Engineering (ICSE '99)*. ACM, New York, NY, USA, 411–420. doi:10.1145/302405.302672
- [12] Bernd Finkbeiner, Felix Klein, and Niklas Metzger. 2022. Live Synthesis. *Innovations in Systems and Software Engineering* 18, 3 (2022), 443–454. doi:10.1007/s11334-022-00447-5
- [13] Carlo Ghezzi, Joel Greenyer, and Valerio Panzica La Manna. 2012. Synthesizing Dynamically Updating Controllers from Changes in Scenario-Based Specifications. In *Proceedings of the 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '12)*. IEEE, Piscataway, NJ, USA, 145–154. doi:10.1109/SEAM

- S.2012.6224401
- [14] Cristiano Giuffrida, Anton Kuijsten, and Andrew S. Tanenbaum. 2013. Safe and Automatic Live Update for Operating Systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*. ACM, New York, NY, USA, 279–292. doi:10.1145/2451116.2451147
 - [15] Christopher M. Hayden, Karla Saur, Edward K. Smith, Michael Hicks, and Jeffrey S. Foster. 2014. Kitsune: Efficient, General-Purpose Dynamic Software Updating for C. *ACM Transactions on Programming Languages and Systems* 36, 4, Article 13 (Oct. 2014), 38 pages. doi:10.1145/2629460
 - [16] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (1990), 463–492. doi:10.1145/78969.78972
 - [17] Takanori Hirano, Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2024. Dynamic Update Controller Synthesis using on-the-fly Exploration. *IEICE Technical Report* 123, 443 (March 2024), 76–78. In Japanese; non-peer-reviewed technical report. <https://ken.ieice.org/ken/paper/20240314Vccd/>
 - [18] Martin Hofmann, Christian Neukirchen, and Harald Rueß. 2016. Certification for μ -Calculus with Winning Strategies. In *Model Checking Software (Lecture Notes in Computer Science, Vol. 9641)*. Springer, Cham, Switzerland, 111–128. doi:10.1007/978-3-319-32582-8_8
 - [19] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering* 16, 11 (1990), 1293–1306. doi:10.1109/32.60317
 - [20] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. doi:10.1023/A:1011254632723
 - [21] Claus Lewerentz and Thomas Lindner. 1995. Case Study “Production Cell”: A Comparative Study in Formal Specification and Verification. In *KORSO: Methods, Languages, and Tools for the Construction of Correct Software*, Manfred Broy and Stefan Jähnichen (Eds.). Lecture Notes in Computer Science, Vol. 1009. Springer, Berlin, Germany, 388–416. doi:10.1007/BFb0015473
 - [22] Xiaoxing Ma, Luciano Baresi, Carlo Ghezzi, Valerio Panzica La Manna, and Jian Lu. 2011. Version-consistent Dynamic Reconfiguration of Component-based Distributed Systems. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ACM, 245–255. doi:10.1145/2025113.2025148
 - [23] Jeff Magee and Jeff Kramer. 2006. *Concurrency: State Models and Java Programs* (2 ed.). John Wiley & Sons, Chichester, UK. <https://www.doc.ic.ac.uk/~jnm/book/>
 - [24] Valerio Panzica La Manna. 2012. Local Dynamic Update for Component-Based Distributed Systems. In *Proceedings of the 15th ACM SIGSOFT Symposium on Component Based Software Engineering (CBSE '12)*. ACM, New York, NY, USA, 167–176. doi:10.1145/2304736.2304764
 - [25] Valerio Panzica La Manna, Joel Greenyer, Carlo Ghezzi, and Christian Brenner. 2013. Formalizing Correctness Criteria of Dynamic Updates Derived from Specification Changes. In *Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '13)*. IEEE, Piscataway, NJ, USA, 63–72. doi:10.1109/SEAMS.2013.6595493
 - [26] Jedidiah McClurg, Hossein Hojjat, Pavol Černý, and Nate Foster. 2015. Efficient Synthesis of Network Updates. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. ACM, New York, NY, USA, 196–207. doi:10.1145/2737924.2737980
 - [27] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. doi:10.1016/j.cosrev.2010.09.009
 - [28] Sahar Mohajerani, Robi Malik, and Martin Fabian. 2014. A Framework for Compositional Synthesis of Modular Nonblocking Supervisors. *IEEE Trans. Automat. Control* 59, 1 (2014), 150–162. doi:10.1109/TAC.2013.2283109
 - [29] MongoDB. [n. d.]. Upgrade to MongoDB Enterprise (Replica Set). MongoDB Database Manual, version 7.0. Accessed 2026-09-29. <https://www.mongodb.com/docs/v7.0/tutorial/upgrade-to-enterprise-replica-set/>
 - [30] Leandro Nahabedian, Victor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2016. Assured and Correct Dynamic Update of Controllers. In *Proceedings of the 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '16)*. ACM, New York, NY, USA, 96–107. doi:10.1145/2897053.2897056
 - [31] Leandro Nahabedian, Victor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2020. Dynamic update of discrete event controllers. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1220–1240. doi:10.1109/TSE.2018.2876843
 - [32] Anas Nooruldeen and Klaus Werner Schmidt. 2015. State Attraction Under Language Specification for the Reconfiguration of Discrete Event Systems. *IEEE Trans. Automat. Control* 60, 6 (2015), 1630–1634. doi:10.1109/TAC.2014.2358811
 - [33] Grigore Roşu and Klaus Havelund. 2005. Rewriting-Based Techniques for Runtime Verification. *Automated Software Engineering* 12, 2 (2005), 151–197. doi:10.1007/s10515-005-6205-y
 - [34] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. 2022. Change Is the Only Constant: Dynamic Updates for Workflows. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New

- York, NY, USA, 350–362. doi:10.1145/3510003.3510065
- [35] The Kubernetes Authors. [n. d.]. Deployments. Kubernetes Documentation. Accessed 2026-09-29. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [36] Sander B. Thuijsman and Michel A. Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Transactions on Embedded Computing Systems* 23, 5, Article 71 (2024), 25 pages. doi:10.1145/3579644
- [37] Stavros Tripakis and Karine Altisen. 1999. On-the-Fly Controller Synthesis for Discrete and Dense-Time Systems. In *Formal Methods (Lecture Notes in Computer Science, Vol. 1708)*. Springer, Berlin, Germany, 233–252. doi:10.1007/3-540-48119-2_15
- [38] Yves Vandewoude, Peter Ebraert, Yolande Berbers, and Theo D’Hondt. 2007. Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates. *IEEE Transactions on Software Engineering* 33, 12 (2007), 856–868. doi:10.1109/TSE.2007.70733
- [39] W. Murray Wonham and Peter J. Ramadge. 1988. Modular Supervisory Control of Discrete-Event Systems. *Mathematics of Control, Signals, and Systems* 1, 1 (1988), 13–30. doi:10.1007/BF02551233
- [40] Ji Zhang and Betty H. C. Cheng. 2006. Using Temporal Logic to Specify Adaptive Program Semantics. *Journal of Systems and Software* 79, 10 (2006), 1361–1369. doi:10.1016/j.jss.2006.02.062